application, REST may receive data in the response it doesn't need. A proper consumer-driven approach, Pact offers a specific wrapper for GraphQL interactions, which we'll explore in chapter 7.

3.8.2 Event-driven systems

Messaging-based communication offers asynchronous capabilities for systems. The messages pass to a message queue consumed by another service that has subscribed to it. The language could be clearer because they also use the term *consumer*. So, explaining the relationship with contract testing could be quite tricky.

Events are a great starting point for implementing contract testing as they are small and concise objects passed around between applications. The only caveat to implementing contract testing with events is the prerequisite that you abstract the message object from the underlying event technology. It's important to separate the code responsible for dealing with the specific tool's protocol level stuff, for example, an AWS lambda handler, the Amazon Web Services (AWS) Simple Notification Service (SNS) topic, and the section of code that handles the message body.

The event-driven contract testing life cycle is shown in figure 3.15 and explained in the following steps:

1. The event consumer publishes the message content as a contract to a broker.
2. The event producer then pulls down the expected message from the contract store broker and verifies that against the message body produced.
3. The event consumer checks the contract verification status in the broker using the can-i-deploy tool.
4. The event consumer deploys the changes.